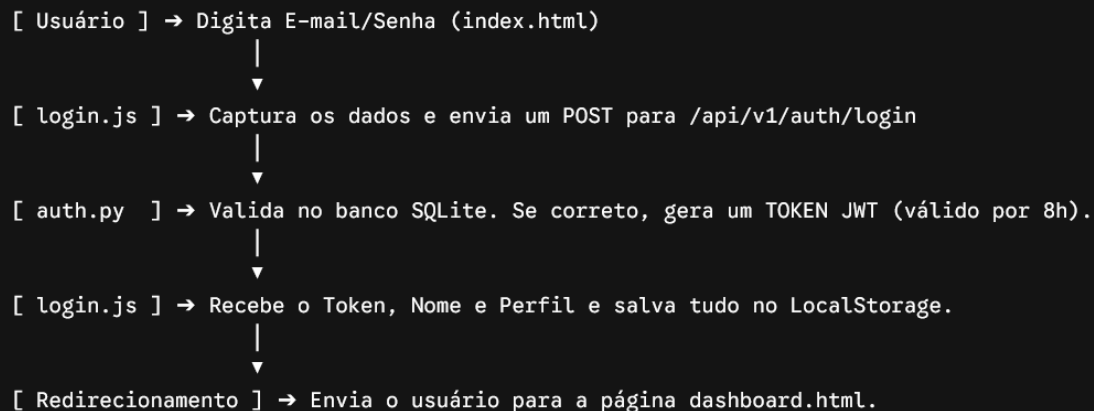


Sumário

Proteção das Páginas no Frontend (auth.js)	2
Como o AgroGestor Carrega as Telas Dinamicamente?	4
O que o safras.py faz (Divisão por Blocos)	6



Proteção das Páginas no Frontend (auth.js)

Para evitar que um usuário mal-intencionado simplesmente digite dashboard.html na barra de endereços sem estar logado, o script auth.js age como um **bloqueador** assim que a página termina de carregar (DOMContentLoaded):

- **Caso 1 (Tentativa de Invasão):** Se o usuário tentar acessar qualquer tela interna (como gerenciamento de talhões ou funcionários) e o Git/Navegador notar que **NÃO** existe o item agro_token salvo no LocalStorage, o script bloqueia o carregamento e o joga imediatamente de volta para a tela de login (index.html).
- **Caso 2 (Já Logado):** Se o usuário já estiver autenticado (agro_token presente) e por acaso voltar para a tela de login (index.html), o script intercepta a ação e o joga de volta para a dashboard.html.
- **Configuração do Operador:** Se o token for válido, ele lê o nome e o nível de acesso (Gestor ou Técnico) guardados no navegador e os escreve dinamicamente na barra lateral do sistema.

A Trava de Segurança no Servidor (security.py)

O security.py é a segurança mais importante do sistema. O frontend esconder telas impede usuários comuns, mas não impede um hacker que saiba usar ferramentas de requisição (como Postman) de tentar acessar ou apagar dados diretamente do banco de dados.

O método get_current_user funciona assim:

1. Sempre que o usuário tenta cadastrar um funcionário, buscar talhões ou ver o painel, a rota do backend exige uma dependência (Depends(oauth2_scheme)).
2. O servidor FastAPI pega o **Token JWT** que o frontend enviou escondido no cabeçalho (*Header Authorization: Bearer*).
3. O script tenta decodificar esse Token usando a assinatura digital única (SECRET_KEY).

4. **Se o token for falso, adulterado ou estiver expirado:** O backend corta a comunicação na hora e devolve um erro 401 Unauthorized (Não Autorizado), protegendo o banco SQLite de qualquer alteração indevida.

Termos Técnicos Explicados de Forma Simples:

- **LocalStorage:** Uma pequena gaveta de memória que o navegador dedica para o seu site lembrar de informações mesmo se o usuário fechar a aba ou atualizar a página.
- **Token JWT:** É como se fosse um "crachá digital criptografado". Ele carrega o ID e o Perfil do usuário de forma que ninguém consiga alterar o conteúdo sem quebrar o código de segurança do servidor.
- **CORS (em main.py):** Uma linha de segurança obrigatória que avisa ao servidor FastAPI (porta 8000) que ele tem permissão para aceitar e responder os cliques vindo das páginas do Frontend (que rodam de forma isolada no navegador).

Pensando no fluxo como uma história real, funciona assim:

1. **A Fábrica (no auth.py):** Quando o usuário faz o login com sucesso, o auth.py pega a sua SECRET_KEY (chave secreta) e o ALGORITHM ("HS256") e "fabrica" o crachá digital (o Token JWT). Ele coloca dentro desse crachá o ID do usuário (sub) e o perfil. O servidor envia esse crachá para o frontend.
2. **O Bolso (no login.js):** O frontend recebe esse crachá e o guarda no localStorage (como se fosse o bolso do usuário). Toda vez que o usuário clica em um botão para gerenciar talhões ou funcionários, o frontend pega esse crachá do bolso e envia junto na requisição.
3. **A Guarita/Inspetor (no security.py):** A requisição chega no backend e cai direto na função get_current_user. O que ela faz? Ela pega aquele mesmo crachá e usa a **mesma** SECRET_KEY e o **mesmo** ALGORITHM para ler o que está escrito dentro dele.

Como o AgroGestor Carrega as Telas Dinamicamente?

A renderização dinâmica do projeto acontece através de **injeção de templates via strings lógicas**, **consumo assíncrono (Fetch API)** e **controle de acesso baseado em papéis (RBAC)**.

1. Injeção Dinâmica por "Gabarito" (Template Strings)

Em vez de reescrever o menu lateral (Sidebar) em todos os arquivos HTML, você usou uma técnica inteligente em components.js:

JavaScript

```
const sidebarHTML = ` <aside> ... ${nomeUsuario} ... </aside> `;  
sidebarContainer.innerHTML = sidebarHTML;
```

- **Como funciona:** O JavaScript cria um "gabarito" do menu em memória. Na hora de renderizar, ele pega as variáveis `${nomeUsuario}` e `${perfilUsuario}` que estavam guardadas no localStorage do navegador e as injeta no meio do HTML antes de jogá-lo na tela. É assim que o nome do operador muda sem precisar de uma página estática para cada um.

2. O Laço de Repetição (forEach) substituindo o HTML manual

Para carregar a lista de safras, as próximas atividades e os alertas, o sistema não usa if-else. Ele usa o método **forEach** (para cada), que lê arrays vindos do banco de dados e gera o HTML dinamicamente na hora.

- **Exemplo Prático em dashboard.js:**

JavaScript

```
dados.proximasAtividades.forEach(atv => {  
  containerAtividades.innerHTML += `<div>... ${atv.nome} ...</div>`;  
});
```

- **A Mágica aqui:** Se o banco de dados retornar 1 atividade, o JavaScript desenha 1 bloco. Se o banco retornar 5 atividades, ele desenha 5 blocos automaticamente, empilhando um abaixo do outro (`+=`). Se não houver nada, aí sim entra um if para mostrar a mensagem estilizada *"Nenhuma atividade planejada"*.

3. Controle de Exibição por Perfil (RBAC - Role-Based Access Control)

O seu código oculta recursos sensíveis baseando-se no cargo de quem está logado. Isso é feito injetando ou removendo classes utilitárias do CSS (Tailwind).

- **Como funciona em components.js e dashboard.js:**

JavaScript

```
if (perfil === 'Técnico Agrícola') {
```

```
const menuFuncionarios = document.getElementById('menu-funcionarios');  
  
if (menuFuncionarios) menuFuncionarios.classList.add('hidden');  
  
}
```

- **O truque:** O HTML do menu de funcionários é injetado, mas se o JavaScript lê no localStorage que o usuário é um **Técnico**, ele adiciona a classe `.hidden` (escondido) do Tailwind. O botão continua no código, mas fica totalmente invisível para o funcionário operacional.

4. Atualização de Propriedades Visuais (Manipulação de Atributos)

Até os gráficos mudam de tamanho sem recarregar a tela. No Card 1 (Progresso da Safra), o círculo verde se mexe mudando uma propriedade geométrica do SVG chamada `stroke-dasharray`.

JavaScript

```
svgProgresso.setAttribute('stroke-dasharray', `${dados.porcentagem}, 100`);
```

Quando o backend calcula que o progresso mudou de 20% para 75%, o JavaScript altera diretamente esse atributo no HTML do gráfico, fazendo com que a barra circular verde cresça de forma responsiva na tela.



Resumo Técnico para memorizar:

O HTML (`dashboard.html`) fornece apenas a "casca vazia" com IDs específicos. O JavaScript (`dashboard.js`) vai até o servidor FastAPI, busca os dados em formato JSON e usa funções de repetição (`forEach`) e manipulação de texto (`innerHTML`) para preencher as lacunas visuais em tempo real.

O que o safras.py faz (Divisão por Blocos)

Ele atende ao requisito de agregação de indicadores através de três operações principais no banco de dados e tratamentos lógicos subsequentes:

CARD 1: Progresso Estatístico da Safra

- **Ação no Banco:** Ele faz uma varredura simplificada (SELECT status) trazendo todas as tarefas associadas àquela safra específica (id_safra = ?).
- **Cálculo Matemático:**
 1. Ele conta a quantidade total de registros retornados utilizando a função len(atividades), armazenando no parâmetro totais.
 2. Ele executa uma filtragem em memória por meio de uma list comprehension para somar apenas os registros cujo status seja rigorosamente igual a 'Realizado' (executadas).
 3. Ele realiza o cálculo da taxa de progresso:

$$\text{porcentagem} = \left(\frac{\text{executadas}}{\text{totais}} \right) \times 100$$

4. **Edge Case (Tratamento de Erro Matemático):** Você adicionou uma trava de segurança (if totais > 0 else 0). Se a safra for nova e não tiver nenhuma atividade cadastrada, o sistema retorna 0% em vez de quebrar a aplicação com um erro de divisão por zero (ZeroDivisionError).

CARD 2: Cronograma e Linha do Tempo Dinâmica

- **Ação no Banco (Agregação por Chave Estrangeira):** Ele executa uma query avançada utilizando um JOIN entre as tabelas atividades e usuarios.
- **Processamento Lógico:**
 - Ele filtra apenas as atividades futuras (status = 'Agendado').
 - Ele substitui o ID numérico do responsável pelo nome real do usuário (u.nome as responsavel) para que a tela mostre o nome legível e não um número sequencial do banco.
 - Ele ordena cronologicamente de forma ascendente (ORDER BY a.data_execucao ASC), garantindo que a tarefa mais urgente apareça primeiro.
 - Ele limita o resultado estritamente a **5 linhas** (LIMIT 5) para evitar sobrecarga visual e quebra do layout no front-end.

CARD 3: Barramento de Análise de Risco (Alertas ESG)

- **Ação no Banco:** Ele consulta a tabela de eventos_extremos buscando incidentes climáticos ou operacionais daquela safra, ordenando dos mais recentes para os mais antigos (ORDER BY data_ocorrência DESC).
- **Processamento Lógico:**
 - Ele converte as linhas puras do banco em um payload padronizado, transformando os campos de texto em mensagens categorizadas de severidade. Ele injeta a propriedade "tipo": "critico" fixamente para disparar os blocos vermelhos de atenção no Tailwind do front-end.

O Payload de Saída (O resultado das contas)

Após processar tudo, o seu código limpa os dados brutos e responde à requisição do front-end com um objeto JSON estruturado contendo exatamente o que as funções do arquivo dashboard.js precisam ler:

JSON

```
{
  "porcentagem": 45,
  "executadas": 9,
  "totais": 20,
  "proximasAtividades": [
    { "data": "2026-06-15", "nome": "Adubação Cobertura", "responsavel": "Técnico Alfa" },
  ],
  "alertas": [
    { "tipo": "critico", "titulo": "OCORRÊNCIA: Chuva de Pedra", "desc": "Registrado em ... : Perda de 5% de área foliar" },
  ]
}
```